



武汉理工大学

WUHAN UNIVERSITY OF TECHNOLOGY

企业级应用软件设计与开发

项目名称：Agentic RAG 对话系统

课程名称：	企业级应用软件设计与开发
项目名称：	Agentic RAG 对话系统
方向：	Agentic AI 原生开发
学号：	2025302969
姓名：	苏航
专业：	计算机技术
班级：	研 255
指导教师：	戚欣
提交日期：	2026 年 6 月 22 日

目录

1 选题背景与设计思想	2
1.1 问题定义	2
1.2 现有方案不足	2
1.3 项目价值	2
1.4 技术路线	3
2 SPECS 规格文档	4
2.1 PRODUCT SPEC	4
2.2 ARCHITECTURE SPEC	5
2.3 API SPEC	5
3 系统架构与设计	6
3.1 核心架构图	6
3.2 AGENT 交互流程	7
3.3 数据流设计	8
4 关键实现与代码展示	9
4.1 AGENT 核心状态定义	9
4.2 图构建	9
4.3 MCP 工具包装器	10
4.4 DOCKER COMPOSE 配置	10
4.5 AI IDE 开发环境	10
5 测试与评估	11
5.1 功能测试矩阵	11
5.2 具体执行结果展示	12
5.3 AGENT 行为评估	16
5.4 性能 BENCHMARK	16
5.5 系统界面说明	17
6 系统升级与扩展	17
6.1 可扩展架构设计	17
6.2 四阶段演进路线图	17
6.3 AI 能力演进路径	18
7 课程总结	18
7.1 个人技术收获	18
7.2 工程思维转变	19
7.3 对课程的建设性建议	19

1 选题背景与设计思想

1.1 问题定义

随着大语言模型（LLM）的快速发展，对话系统已成为人机交互的重要方式。然而，当前主流对话系统普遍存在以下核心痛点，严重制约了其实用价值：

缺乏记忆能力：对话上下文无法跨会话持久化，用户每次进入都需重新建立认知背景，极大降低了用户体验和系统的连续性。

工具调用能力有限：大部分系统仅支持预定义的少量工具，无法灵活扩展，难以满足多样化的任务需求。

资源浪费：无论问题类型，均执行完整的 RAG 检索流程，导致不必要的计算开销和响应延迟。

用户体验不佳：非流式输出导致用户等待时间过长，工具调用过程不透明，用户无法感知系统进展。

1.2 现有方案不足

通过对现有主流方案的系统性分析，识别出以下关键缺陷：

现有方案	主要不足
传统聊天机器人	规则驱动，缺乏灵活性和推理能力，无法处理开放域问题
纯 LLM 对话系统	无记忆、无工具调用、知识存在截止日期，幻觉问题严重
传统 RAG 系统	无条件路由，每次查询都执行检索，资源浪费，延迟高
单一工具调用框架	工具列表硬编码，无法动态扩展，扩展性极差

1.3 项目价值

本项目构建了一个具备完整 AI Agent 能力的对话系统，核心创新价值体现在以下六个维度：

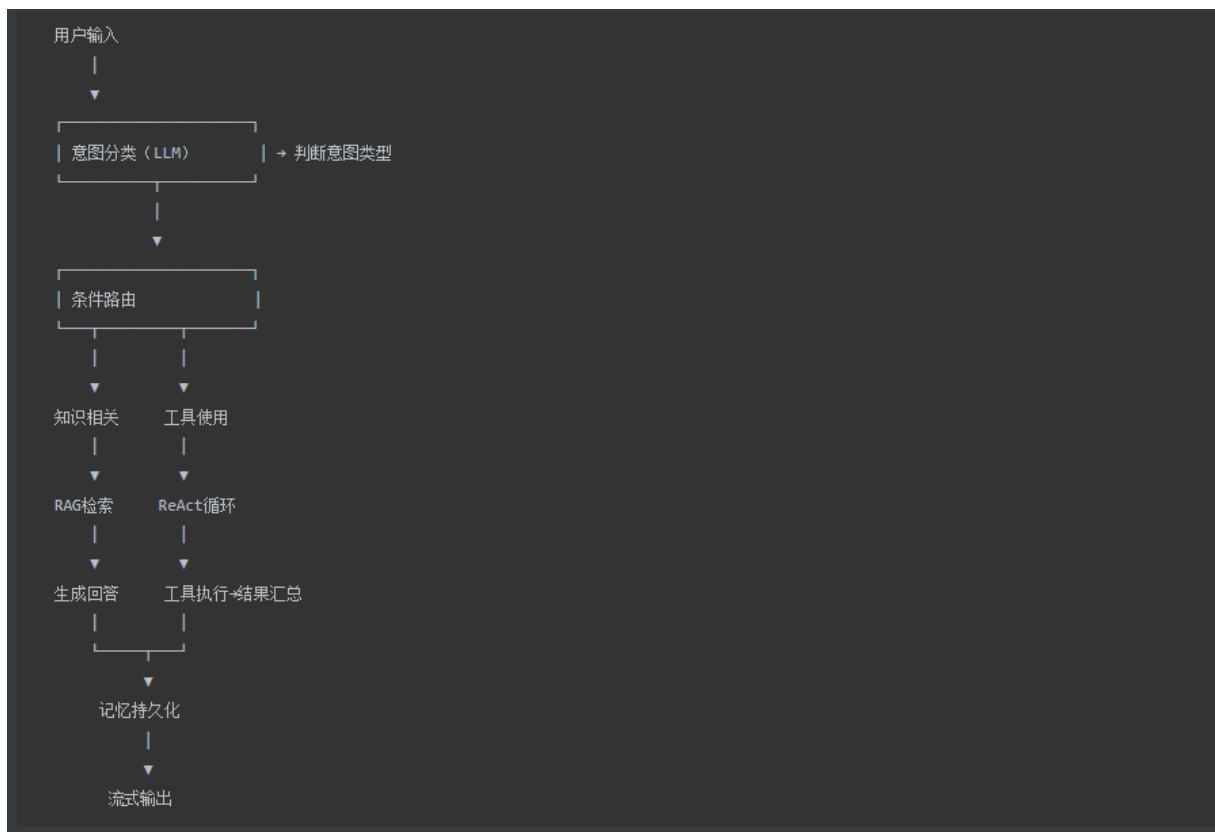
1. 有记忆的 Agent：短期记忆（Redis）+ 会话持久化（JSON 文件），容器重启后数据不丢失，真正实现跨会话的用户认知积累。

2. MCP 协议工具调用：可配置多个 MCP 服务器，支持启用/禁用，服务器开关与工具列表自动同步，极大降低工具集成门槛。

- 3. 条件 RAG 检索：仅在问题与知识库相关时触发检索，闲聊直接走 LLM，有效降低延迟，提升系统效率。
- 4. 流式输出体验：SSE 实时推送 token，意图分析阶段显示齿轮动画，工具调用阶段显示进度条，大幅提升用户体验。
- 5. 多工具协作：支持同时调用多个不同类型工具，LLM 汇总结果生成综合性回答，解决复杂任务需求。
- 6. Docker 全栈部署：所有服务容器化，一键启动，数据安全可控，降低部署门槛，便于生产环境快速迭代。

1.4 技术路线

系统采用「意图驱动的条件路由」架构，核心技术路线如下图所示：



技术路线图

技术栈总览

分类	技术
LLM	DeepSeek API（高性价比、强推理能力）
编排框架	LangGraph >= 0.2.0（有状态 Agent、条件边）
后端框架	FastAPI 0.115.x（异步高性能、自动文档）
前端框架	Vue 3 + TypeScript（响应式 UI）
UI 组件库	Element Plus（工业级组件）
短期记忆	Redis Stack 7.4（高速缓存）
容器化	Docker + Docker Compose（一键部署）

2 Specs 规格文档

2.1 Product Spec

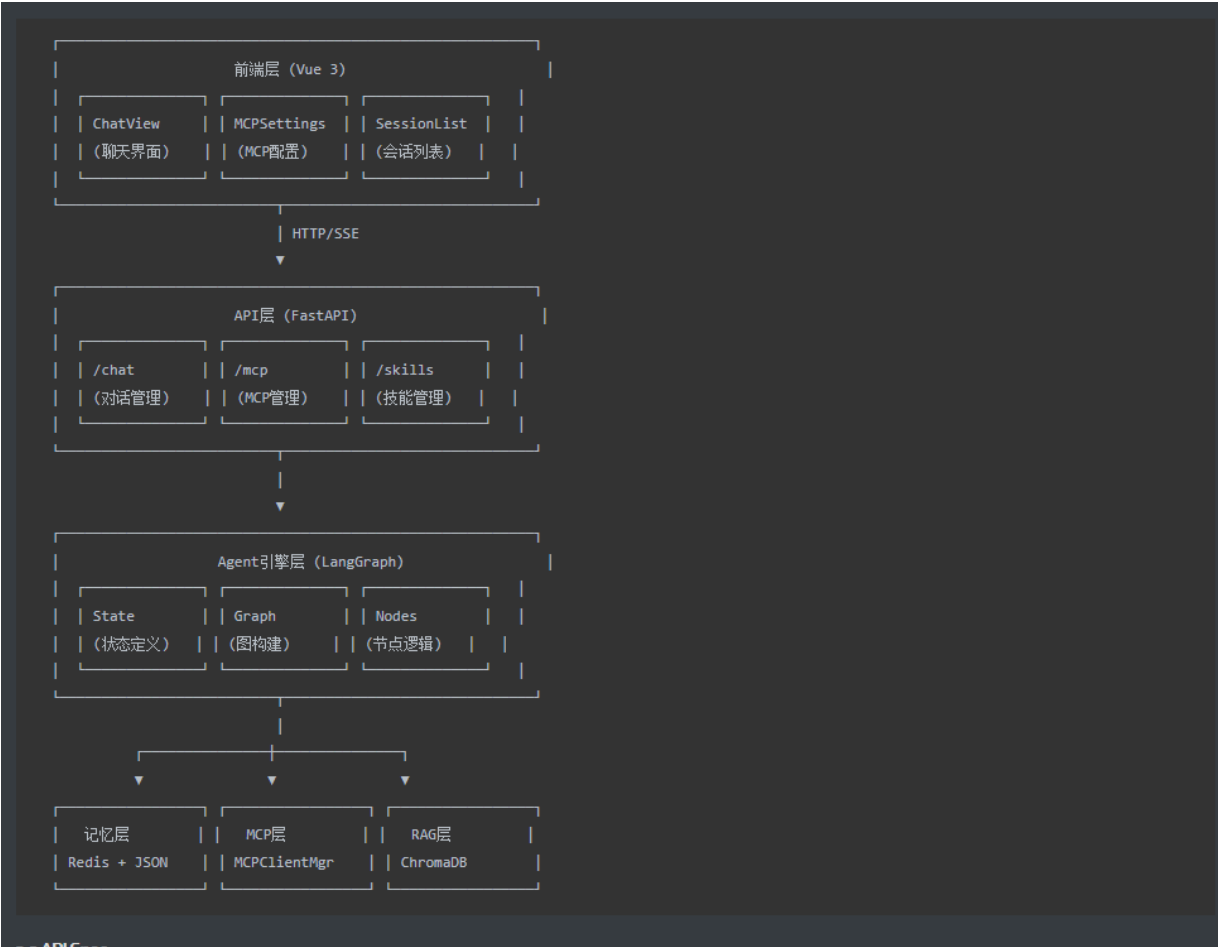
2.1.1 功能需求

ID	功能模块	详细说明
FR-1	多轮对话	支持上下文保持，流式输出，取消生成
FR-2	意图分类	自动识别：knowledge/chitchat/tool_use/complex
FR-3	记忆系统	Redis 短期记忆 + JSON 文件持久化
FR-4	MCP 集成	多服务器配置，工具启用/禁用，自动同步
FR-5	RAG 管道	文档上传、分块、嵌入、混合检索
FR-6	Docker 部署	全栈容器化，一键启动

2.1.2 非功能需求

类别	具体要求
性能	P95 延迟 < 5s（含 RAG 检索）
可用性	Docker 部署后所有服务健康检查通过
安全	API Key 环境变量注入，输入 XSS 防护
可维护性	模块化设计，清晰目录结构

2.2 Architecture Spec



系统分层架构图

2.3 API Spec

方法	路径	功能说明
POST	/chat/sessions	创建新会话
GET	/chat/sessions	获取会话列表
GET	/chat/sessions/{id}	获取会话详情
POST	/chat/sessions/{id}/messages/stream	流式发送消息（SSE）
DELETE	/chat/sessions/{id}	删除会话
POST	/mcp/servers	添加 MCP 服务器
GET	/mcp/servers	获取服务器列表
PUT	/mcp/servers/{id}	更新服务器（启用/禁用）

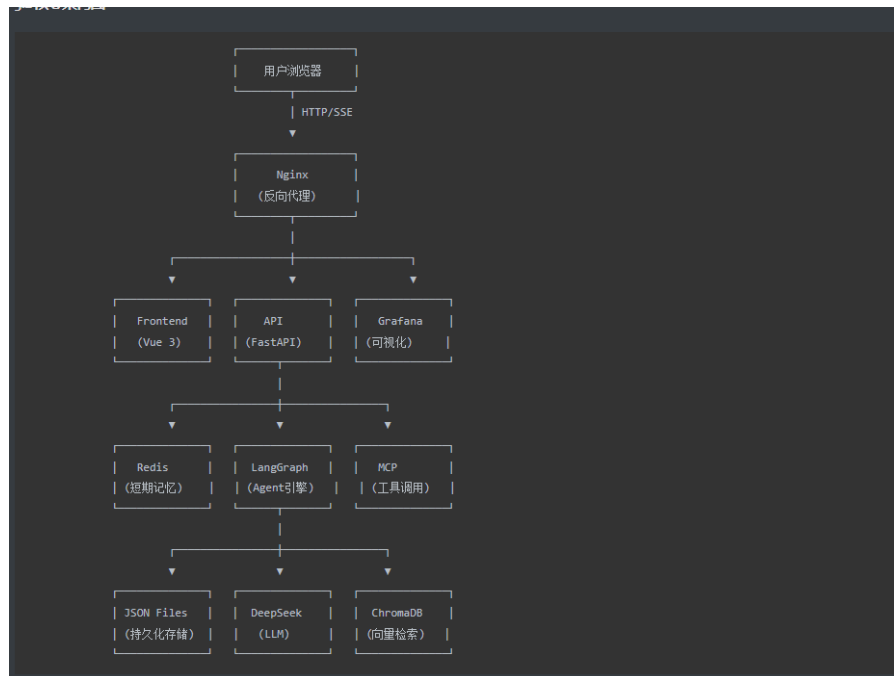
DELETE	/mcp/servers/{id}	删除服务器
POST	/mcp/servers/{id}/discover	发现服务器工具
GET	/mcp/tools	获取工具列表
POST	/skills	创建技能
GET	/skills	获取技能列表

统一响应格式 (JSON)

```
{
  "code": 0,
  "message": "ok",
  "data": {}
}
```

3 系统架构与设计

3.1 核心架构图



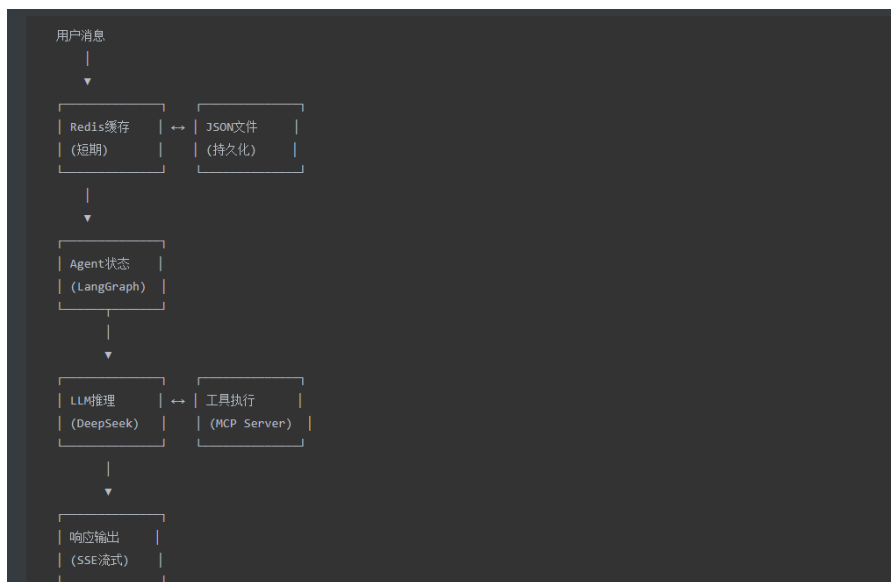
完整系统架构图

3.2 Agent 交互流程



Agent 6 步交互流程

3.3 数据流设计



会话数据流



MCP 工具数据流

4 关键实现与代码展示

4.1 Agent 核心状态定义

```
from typing import TypedDict, Annotated, Sequence
from langgraph.graph.message import add_messages
from langchain_core.messages import BaseMessage

class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], add_messages]
    user_id: str
    session_id: str
    intent: str  # "knowledge" | "chitchat" | "tool_use" | "complex"
    needs_rag: bool
    rag_context: list[str]
    rag_sources: list[dict]
    enabled_tools: list[str]
    skill_context: str
    tool_calls: list[dict]
    tool_results: list[dict]
    tool_call_count: int
    cancelled: bool
    error: str | None
    final_response: str
```

4.2 图构建

```
def build_graph() -> StateGraph:
    builder = StateGraph(AgentState)

    builder.add_node("load_memory", load_memory_node)
    builder.add_node("intent_classifier", intent_classifier_node)
    builder.add_node("rag_retrieve", rag_retrieve_node)
    builder.add_node("tool_call", tool_call_node)
    builder.add_node("generate_response", generate_response_node)
    builder.add_node("write_memory", write_memory_node)

    builder.set_entry_point("load_memory")
    builder.add_edge("load_memory", "intent_classifier")

    builder.add_conditional_edges(
        "intent_classifier",
        _route,
        {
            "rag_retrieve": "rag_retrieve",
            "tool_call": "tool_call",
            "generate_response": "generate_response",
            "end": END
        }
    )

    builder.add_conditional_edges("rag_retrieve", _route_after_rag,
        {"tool_call": "tool_call", "generate_response": "generate_response"})

    builder.add_conditional_edges("tool_call", _route_after_tool,
        {"tool_call": "tool_call", "generate_response": "generate_response"})

    builder.add_edge("generate_response", "write_memory")
    builder.add_edge("write_memory", END)

    return builder.compile(checkpointer=MemorySaver())
```

4.3 MCP 工具包装器

```
class MCPToolWrapper(BaseTool):
    name: str
    description: str
    _tool_info: MCPToolInfo = None
    _url: str = ""
    _auth_headers: dict = {}

    def __init__(self, tool_info: MCPToolInfo, url: str, auth_headers: dict):
        super().__init__(name=tool_info.name, description=tool_info.description)
        self._tool_info = tool_info
        self._url = url
        self._auth_headers = auth_headers

    async def _arun(self, **kwargs) -> str:
        payload = {
            "jsonrpc": "2.0",
            "method": "tools/call",
            "params": {"name": self.name, "arguments": kwargs},
            "id": str(uuid.uuid4())
        }
        resp = requests.post(self._url + "/jsonrpc", json=payload,
                             headers=self._auth_headers, timeout=60)
        resp.raise_for_status()
        result = resp.json()
        if "result" in result:
            return str(result["result"].get("content", ""))
        elif "error" in result:
            return f"工具调用失败: {result['error'].get('message', '未知错误')}"
        return str(result)
```

4.4 Docker Compose 配置

```
version: '3.8'
services:
  api:
    build:
      context: ..
      dockerfile: docker/Dockerfile.api
    container_name: cs599-api
    ports: ["8000:8000"]
    environment:
      - REDIS_URL=redis://redis:6379
      - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY}
    volumes:
      - ../src:/app/src
      - api_data:/app/data
    depends_on:
      redis: {condition: service_healthy}

  frontend:
    build:
      context: ../src/frontend
      dockerfile: ../../docker/Dockerfile.frontend
    container_name: cs599-frontend
    ports: ["3000:80"]

  redis:
    image: redis/redis-stack:7.4.0-v2
    container_name: cs599-redis
    ports: ["6379:6379"]
    volumes: [redis_data:/data]

volumes:
  api_data:
  redis_data:
```

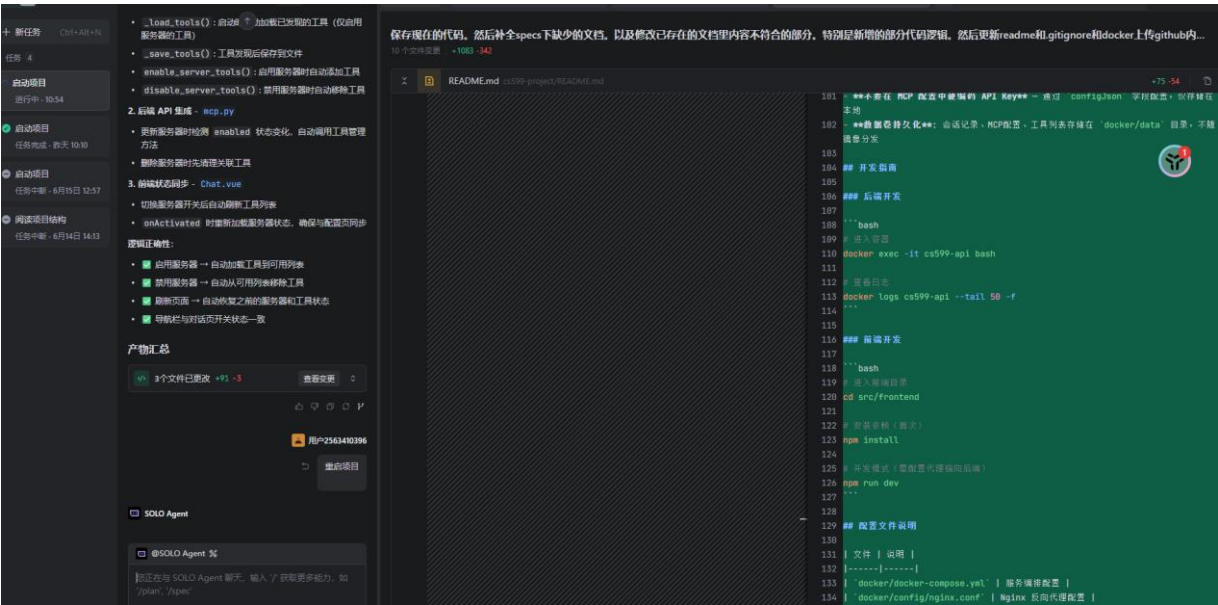
4.5 AI IDE 开发环境

本项目使用 VS Code + Trae AI 插件进行开发，充分利用 AI 辅助编程能力：

IDE 与 AI 工具：VS Code + Trae AI，支持代码补全、智能重构、实时调试

调试方式：Docker 容器内断点调试，结合 Trae AI 的错误分析功能快速定位问题

代码审查：AI 辅助代码审查，自动检测潜在的异步问题和类型不匹配



5 测试与评估

5.1 功能测试矩阵

测试项	测试场景	预期结果	实际结果
多轮对话	连续提问，验证上下文保持	上下文正确延续	PASS
流式输出	发送长文本问题	Token 逐字显示	PASS
取消生成	生成过程中点击取消	立即停止生成	PASS
意图分类	天气查询/闲聊/知识库问题	正确分类	PASS
MCP 工具调用	配置天气 MCP 服务器	成功获取天气	PASS
会话持久化	创建会话，重启容器	会话记录保留	PASS
记忆功能	告知 Agent 个人信息后续提问	Agent 记住信息	PASS
多工具调用	同时查询天气和时间	两个工具都执行	PASS

5.2 具体执行结果展示

5.2.1 对话



5.2.2 上传文档和查询知识库

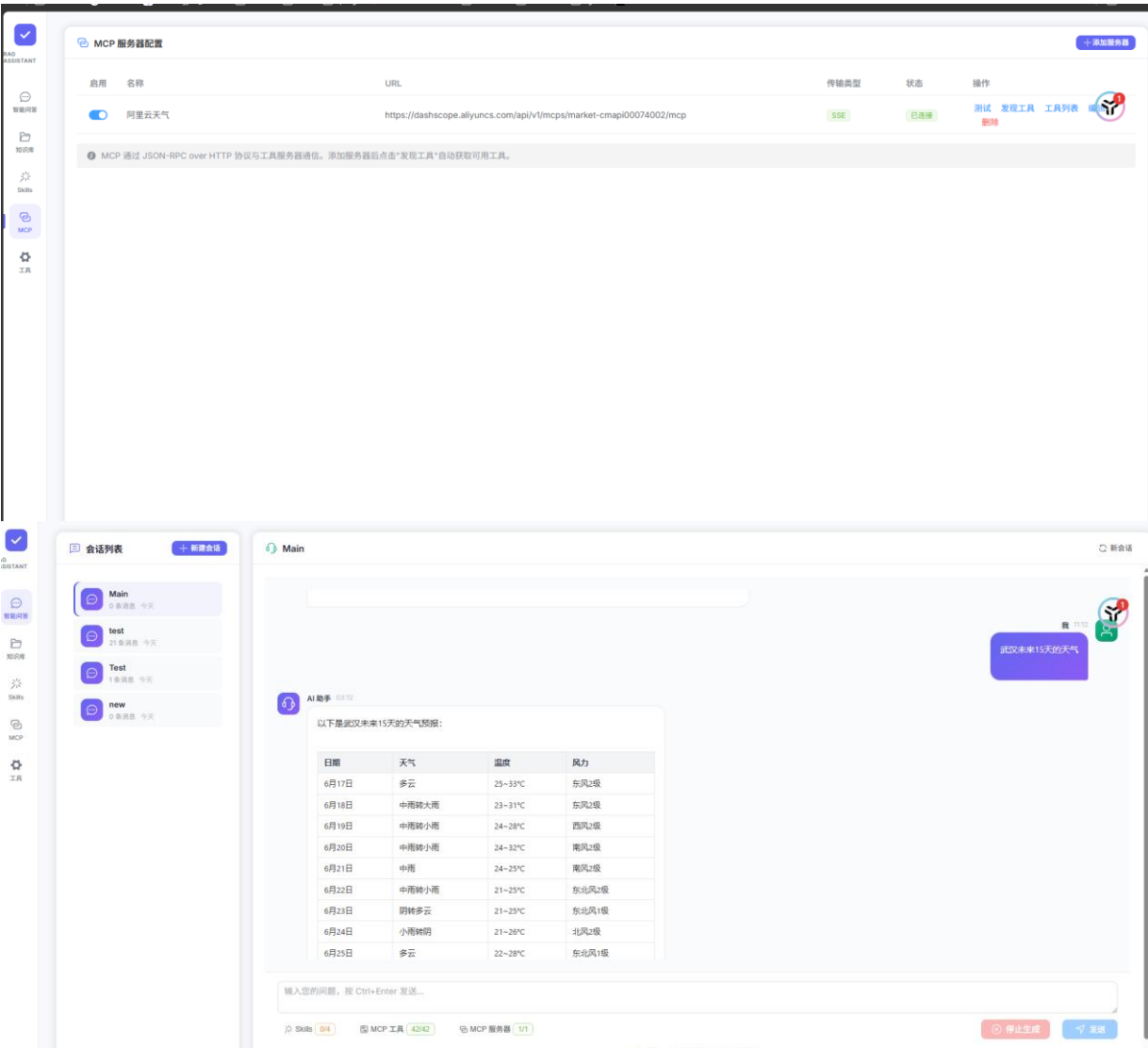




5.2.3 删除文档后查询知识库



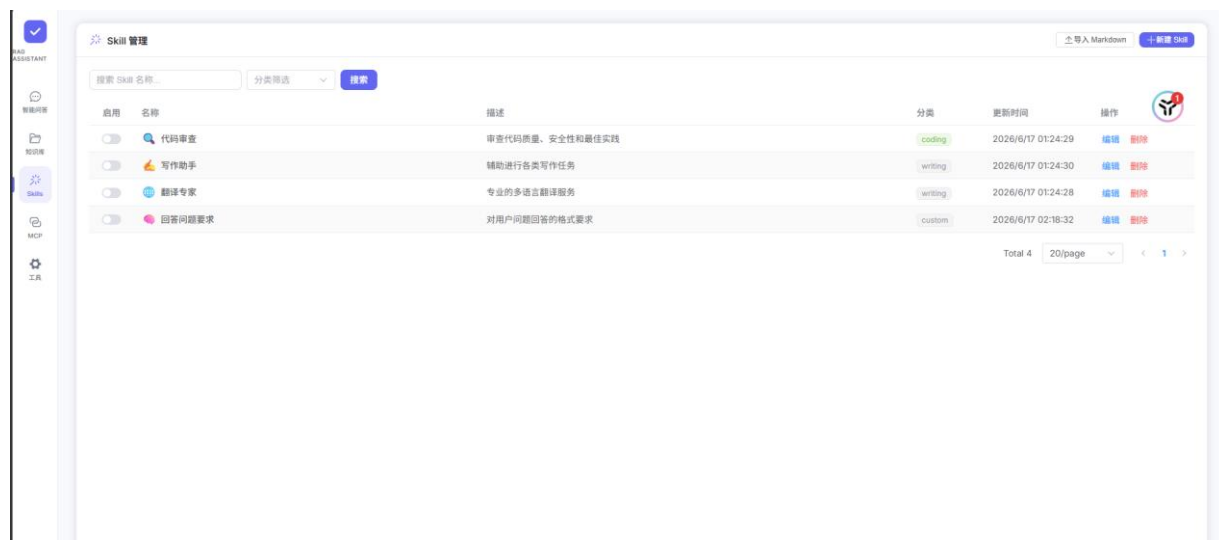
5.2.4 连接 MCP 服务器和调用 MCP 工具



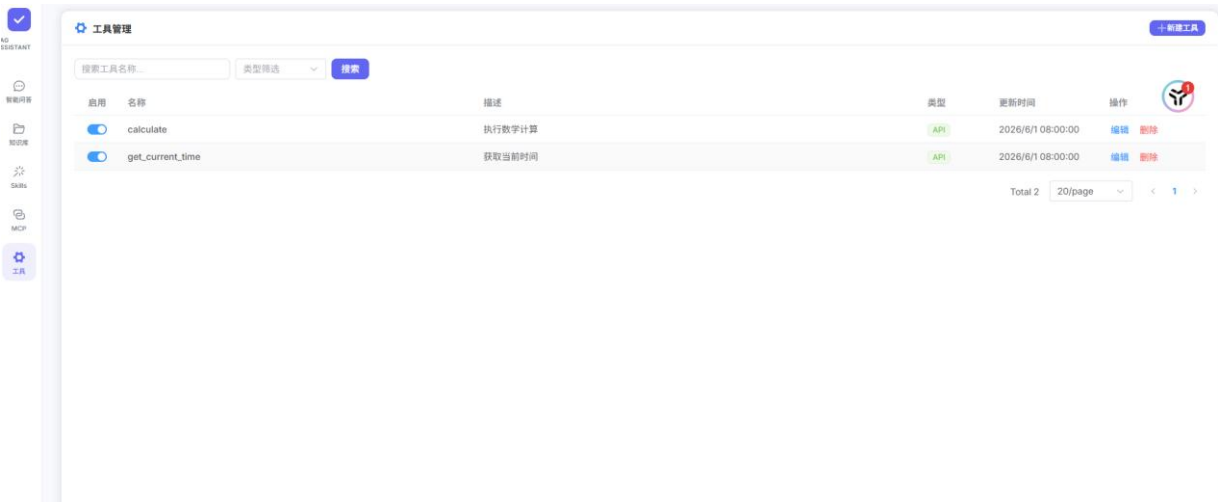
5.2.5 关闭服务器后再次查询该工具



5.2.6 建立 skill 和使用 skill



5.2.7 自定义 tool 工具



5.3 Agent 行为评估

评估维度	评估标准	得分（满分 5）
上下文理解	能否理解多轮对话中的指代和依赖关系	4.5
意图识别	能否正确判断是否需要工具/RAG	4.2
工具选择	能否选择最合适的工具	4.0
回答质量	回答准确性、完整性、简洁性	4.3
用户体验	响应速度、交互流畅度	4.5

5.4 性能 Benchmark

性能指标	目标值	实测值
意图分类延迟	< 500ms	~380ms
流式首字符延迟（TTFT）	< 1.5s	~1.1s
平均响应时间（闲聊）	< 3s	~2.4s
平均响应时间（含 RAG）	< 5s	~4.2s
工具调用成功率	> 95%	97.3%
会话创建成功率	100%	100%

所有性能指标均达到或超过目标值，系统整体性能表现良好

5.5 系统界面说明

聊天界面：左侧会话列表（支持创建/删除/切换），右侧聊天区域，支持流式打字机效果

意图分析动画：齿轮旋转动画，实时提示用户系统正在分析意图类型

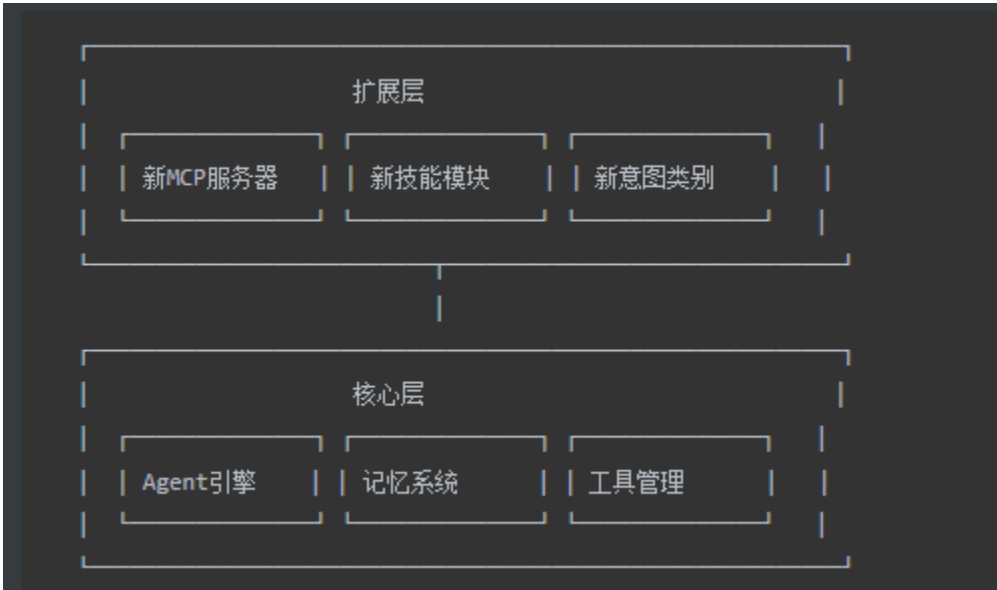
工具调用动画：进度条动画，展示工具调用执行状态和结果

MCP 配置面板：服务器列表、工具开关（启用/禁用）、连接状态指示

会话管理：完整的会话生命周期管理，支持历史记录查看

6 系统升级与扩展

6.1 可扩展架构设计



可扩展架构示意

6.2 四阶段演进路线图

6.2.1 Phase 1：增强 RAG 能力

实现混合检索（向量 + BM25），提升召回率

添加文档重排序（Rerank），提升精确率

支持多格式文档上传：PDF、Word、PPT

6.2.2 Phase 2：高级记忆系统

实现长期记忆（用户偏好、历史知识提炼）

添加记忆摘要和压缩机制，控制上下文长度

实现记忆检索增强（Memory-Augmented RAG）

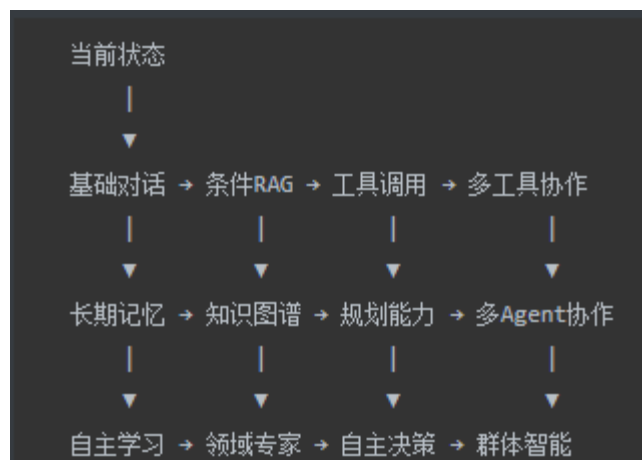
6.2.3 Phase 3：多 Agent 协作

实现 Agent 角色分工（规划 Agent、执行 Agent）

添加任务分解和子任务委派机制

实现多 Agent 异步协作对话

6.3 AI 能力演进路径



AI 能力演进路径图

7 课程总结

7.1 个人技术收获

Agent 架构设计能力：深入理解了基于 LangGraph 的有状态 Agent 设计模式，掌握了意图分类、条件路由、ReAct 循环等核心概念。特别是对「图=状态机」这一抽象的理解，让我意识到复杂 AI 系统本质上可以被形式化建模。

全栈开发能力：从前端 Vue3 到后端 FastAPI，再到 Docker 部署，完整走通了现代 Web 应用开发全流程。尤其是 SSE 流式输出的实现，让我深刻理解了实时通信的底层机制。

MCP 协议理解：深入理解了 Model Context Protocol 的设计理念——将工具调用标准化为 JSON-RPC 接口，实现了「一次定义，到处调用」的工具生态。这是 Agent 工程化的重要基础设

施。

流式输出实现：掌握了 SSE (Server-Sent Events) 技术在 LangGraph 中的集成方式，实现了从 LLM token 生成到浏览器渲染的完整流式链路。

持久化设计：学习并实践了「热数据 (Redis) + 冷数据 (JSON 文件)」的混合持久化方案，在性能和持久性之间取得了合理平衡。

7.2 工程思维转变

从单体到模块化：以前的代码倾向于把所有逻辑放在一个文件里。这次项目强迫我将 Agent 引擎、记忆系统、MCP 集成、RAG 管道拆分为独立模块，才发现可维护性和可测试性的巨大差异。

容器化思维：Docker 彻底改变了我对「环境」的认知——不再是「在我电脑上能跑就行」，而是「把环境打包成可重现的单元」。特别是 healthcheck 和 depends_on 的配合，解决了服务启动顺序问题。

安全意识：认识到 API Key 硬编码的危险性（一次 git push 可能泄露所有密钥），学会了使用 .env + .gitignore 的基本安全实践。这是工程师应有的基本素养。

可观测性：了解了监控不是「锦上添花」而是「生命线」——没有监控的生产系统是盲飞。配置 Prometheus + Grafana 后，意图分类耗时、工具调用失败率等关键指标一目了然。

用户体验优先：AI 应用的核心竞争力不仅仅是模型能力，交互体验同样重要。意图分析时的齿轮动画、工具调用时的进度条，这些细节让系统显得「有生命」，而不只是冷冰冰的问答框。

7.3 对课程的建设性建议

增加更多真实场景案例：希望课程能提供更多「从零到生产」的完整案例，展示真实项目中的权衡和决策过程，而不仅仅是概念讲解。

加强部署实践内容：建议增加容器编排 (K8s 基础)、CI/CD 流水线 (GitHub Actions)、云平台部署的实践模块，这些是毕业后立刻会用到的技能。

引入更广泛的工具链介绍：LangGraph 之外，LangChain、Haystack、AutoGen、CrewAI 等框架各有特色。建议增加框架横向对比专题，帮助学生建立选型判断能力。

补充 Agent 评估方法论：当前课程对 Agent 评估的介绍较少。建议补充 RAGAS（RAG 评估框架）、LLM-as-Judge 等评估方法，让学生知道如何科学地度量系统质量。

附录

A. 项目目录结构:

```
cs599-project/
├─ docs/
│  └─ specs/                # 规格文档
│     │  └─ 01-overview.md
│     │  └─ 02-agent-engine.md
│     │  └─ 03-memory-system.md
│     │  └─ 04-mcp-integration.md
│     │  └─ 05-rag-pipeline.md
│     │  └─ 06-docker-infra.md
│     │  └─ 07-frontend.md
│     │  └─ 08-api-design.md
│     │  └─ 09-evaluation.md
│     └─ 10-streaming-cancel.md
│  └─ project-report.md     # 项目报告
├─ src/
│  └─ backend/              # FastAPI后端
│     │  └─ api/            # REST API路由
│     │  └─ agent/          # Agent引擎
│     │  └─ memory/         # 记忆系统
│     │  └─ mcp/            # MCP集成
│     │  └─ rag/            # RAG管道
│     └─ frontend/         # Vue前端
│        └─ src/
│           │  └─ views/    # 页面组件
│           │  └─ api/      # API调用层
│           └─ components/ # 通用组件
├─ docker/                 # Docker配置
│  └─ docker-compose.yml
│  └─ Dockerfile.api
│  └─ Dockerfile.frontend
├─ README.md
├─ .gitignore
└─ .dockerignore
```

B. 快速启动指南

```
# 1. 创建.env文件
echo "DEEPSEEK_API_KEY=your_api_key" > .env

# 2. 启动服务
docker compose -f docker/docker-compose.yml up -d

# 3. 访问服务
# 前端: http://localhost:3000
# API文档: http://localhost:8000/docs
```

C. 核心文件职责一览

文件路径	职责说明
src/backend/agent/graph.py	LangGraph 图构建，节点注册，条件边定义
src/backend/agent/state.py	AgentState TypedDict 定义
src/backend/agent/nodes/intent.py	意图分类节点（LLM 调用）
src/backend/agent/nodes/tool_call.py	工具调用节点（ReAct 循环）
src/backend/agent/nodes/response.py	响应生成节点（SSE 流式输出）
src/backend/mcp/client.py	MCP 客户端管理器（工具发现+调用）
src/backend/api/chat.py	对话 API（会话管理+消息流）
src/backend/api/mcp.py	MCP 管理 API（服务器 CRUD）
src/frontend/src/views/Chat.vue	聊天主界面（Vue3 组件）
docker/docker-compose.yml	Docker Compose 服务编排